Cycle

Resumo do capítulo

SOLID e Design Patterns: Conceitos e aplicabilidade

O desenvolvimento de software moderno exige mais do que apenas a habilidade de escrever código funcional; ele demanda uma compreensão profunda de princípios de design e padrões arquiteturais que orientem a criação de sistemas robustos, adaptáveis e fáceis de manter. Este capítulo tem como objetivo explorar esses conceitos cruciais, focando nos princípios SOLID e em design patterns, elementos essenciais para construir software de alta qualidade. Ao longo desta discussão, utilizaremos um exemplo prático de um sistema de emissão de notas fiscais, que servirá como um fio condutor para demonstrar a aplicação desses conceitos na prática, e que servirá como um exemplo para diversos cenários.

O objetivo central deste material é apresentar os princípios SOLID e os design patterns como ferramentas essenciais para a criação de software com um design superior, indo além das meras definições teóricas. Através de um contexto prático, buscamos demonstrar como essas ferramentas de design de software contribuem para a construção de um código mais adaptável, flexível e de fácil manutenção. Utilizaremos um projeto como guia para ilustrar a aplicação desses princípios, padrões e ferramentas de arquitetura de software, destacando como os Design Patterns e os princípios SOLID se complementam e, quando aplicados corretamente, são cruciais para o desenvolvimento de software de alta qualidade.

O projeto que utilizaremos ao longo deste capítulo é um sistema de emissão de notas fiscais de serviço, que simula um cenário real de uma aplicação de negócio, mas que tem como objetivo principal demonstrar a aplicabilidade dos conceitos que serão apresentados ao longo do material. Este projeto aborda diferentes regimes contábeis, como o regime de caixa e o regime de competência, para ilustrar como as decisões de design podem influenciar a forma como a aplicação se adapta a diferentes necessidades e requisitos. Embora o projeto seja uma simulação e não represente todas as complexidades de um sistema real de emissão de notas fiscais, a sua complexidade foi cuidadosamente planejada para permitir a aplicação dos padrões de design e princípios SOLID de forma didática, com o objetivo de demonstrar como o uso de diferentes ferramentas e padrões de design podem simplificar e tornar mais flexível o código, e qu

esses conceitos podem ser aplicados independente da tecnologia usada para a criação do software.

Principais pontos abordados no projeto:

- **Separação de Responsabilidades:** O código é organizado em camadas distintas (domínio, aplicação e infraestrutura), cada uma com sua responsabilidade específica, garantindo que cada parte do sistema trate de um aspecto particular do problema, e que mudanças sejam restritas à cada uma das camadas.
- **Desacoplamento:** A dependência entre componentes é minimizada através da injeção de dependências, do uso de interfaces e do padrão *Mediator*, criando um sistema modular onde alterações em uma parte não afetem outras partes da aplicação.
- **Reutilização:** O uso de design patterns como *Adapter*, *Strategy* e *Decorator* permite a criação de componentes reutilizáveis e adaptáveis a diferentes situações e contextos de uso.
- **Flexibilidade:** A utilização de padrões como *Factory*, e principalmente o *Strategy*, que define uma família de algoritmos intercambiáveis, promove a adaptação a diferentes necessidades, sem que o software precise ser reescrito ou que a aplicação precise conhecer as subclasses concretas de cada tipo de contrato.
- **Extensibilidade:** O uso do padrão *Decorator*, permite adicionar novas funcionalidades de forma dinâmica sem modificar a estrutura original dos objetos. O *Open/Closed Principle*, que diz que o software precisa estar aberto a extensões e fechado a modificações, foi respeitado através de diversas interfaces e abstrações que permitem a evolução do sistema sem que haja a necessidade de mudanças no código já existente.

Ao final da leitura, você terá uma visão geral sobre o mundo dos design patterns e dos princípios SOLID, com uma compreensão detalhada dos padrões mais utilizados no mercado, através da aplicação em um cenário prático. Embora o projeto ilustre a utilização de alguns padrões, é importante que você também esteja ciente da existência de outros padrões que são igualmente relevantes, e que podem ser utilizados em contextos específicos, e que também serão apresentados de forma resumida. Ao entender a aplicabilidade de cada padrão, você poderá escolher as soluções mais adequadas para cada cenário, e não aplicar os padrões de forma arbitrária ou sem um contexto de uso.

Contexto Histórico e a Importância dos Design Patterns

Os design patterns surgiram como respostas a problemas recorrentes no desenvolvimento de software. Em vez de reinventar a roda a cada projeto, os desenvolvedores passaram a identificar e codificar soluções eficazes e reutilizáveis. No final dos anos 1980 e início de 1990, o termo design pattern começou a se popularizar,

principalmente por meio do trabalho do Gang of Four (GoF), que catalogou diversos padrões que se tornaram referência na área.

O estudo dos design patterns não é apenas uma jornada teórica, mas uma necessidade prática para todo desenvolvedor que busca criar software de alta qualidade. A aplicação dos padrões de design pode simplificar a construção de sistemas complexos, reduzir custos de manutenção e garantir que o código seja mais extensível e fácil de entender. Padrões como Adapter, Strategy e Decorator facilitam a criação de sistemas que podem ser adaptados a diferentes situações sem a necessidade de grandes mudanças no código base.

A Relação Intrínseca entre Design e Arquitetura em Software

No mundo do desenvolvimento de software, o design e a arquitetura são como duas faces de uma mesma moeda, cada uma desempenhando um papel vital na construção de sistemas robustos, flexíveis e eficazes. Eles não são conceitos isolados, mas sim forças interdependentes que moldam e são moldadas pelo processo de criação de software. Enquanto a arquitetura estabelece o esqueleto e a estrutura global do sistema, o design se concentra nos detalhes que dão vida a essa estrutura. A relação entre ambos é fundamental para o sucesso de um projeto, e a compreensão dessa dinâmica é indispensável para todo profissional de desenvolvimento de software.

Arquitetura: As Decisões de Alto Nível

A arquitetura de software define as decisões fundamentais que guiarão o desenvolvimento. Essas decisões incluem a escolha da linguagem de programação, que impõe um estilo particular de codificação e influência a escolha de paradigmas de design, e de frameworks que fornecem a base para a construção da aplicação, definindo a estrutura, os padrões e as bibliotecas a serem utilizadas.

Além disso, a arquitetura define o paradigma de desenvolvimento, que pode ser orientado a objetos (OO), funcional ou procedural. Cada um desses paradigmas possui suas próprias abordagens para a organização e manipulação de dados e de fluxos de controle, o que afeta diretamente as escolhas de design. Por exemplo, uma arquitetura baseada em microsserviços impõe uma abordagem de design em que os componentes da aplicação são distribuídos e independentes, enquanto uma arquitetura monolítica favorece um design mais centralizado, com a alocação de diversas responsabilidades dentro de uma única aplicação. As decisões arquitetônicas, portanto, fornecem o contexto dentro do qual o design se desenrola.

Design: Dando Vida à Arquitetura

O design de software se concentra nas decisões de baixo nível, que detalham como as classes, os objetos, os componentes e os módulos são organizados e interagem entre si. É aqui que os design patterns desempenham um papel fundamental, oferecendo soluções comprovadas para problemas comuns. O design engloba a definição das responsabilidades, interfaces, a escolha de algoritmos e como os dados são estruturados, o tratamento de exceções e como as funcionalidades serão testadas.

A qualidade do design é essencial para garantir que o software seja fácil de entender, modificar e manter. Um design bem elaborado também facilita a implementação de requisitos complexos e promove a extensibilidade do sistema. O design define como a complexidade é gerida, como as preocupações são separadas e como cada componente do software contribui para um objetivo maior.

A Influência Recíproca entre Arquitetura e Design

É essencial compreender que a arquitetura restringe as possibilidades de design. Por exemplo, a escolha de uma linguagem de programação procedural como C, impõe um design no qual o código é executado em sequência, dificultando a implementação de padrões que se baseiam em orientação a objetos, como o padrão Strategy. Já a escolha de uma linguagem orientada a objetos como Java ou C# possibilita a aplicação de padrões que envolvem conceitos como herança, polimorfismo e encapsulamento. Do mesmo modo, ao decidir utilizar uma linguagem funcional como Haskell ou Clojure, pode encorajar padrões que priorizam a composição de funções.

Por outro lado, o design pode influenciar a arquitetura. Por exemplo, ao identificar a necessidade de um sistema com alta taxa de escalabilidade e tolerância a falhas, pode-se adotar uma arquitetura baseada em microsserviços que suporta o desacoplamento e a flexibilidade desejada. A escolha de um padrão de comunicação específico, como um message broker, também afeta o design dos componentes do sistema, que precisam ser construídos para lidar com comunicação assíncrona. Portanto, arquitetura e design não são decisões sequenciais, mas iterativas, que se ajustam e se refinam ao longo do ciclo de vida do projeto.

A Natureza da Arquitetura e do Design

É importante notar que paradigmas de desenvolvimento como programação procedural, orientação a objetos e programação funcional são abordagens para a organização do código em tempo de desenvolvimento, e fazem parte do que pode ser chamado de *design de software*, não de arquitetura. A arquitetura, por sua vez, trata de decisões de mais alto nível, que envolvem a estrutura geral do sistema, como o tipo de persistência de dados, o tipo de comunicação, o tipo de interface, entre outras decisões. Embora a escolha do paradigma de programação influencie o design, e, consequentemente, a arquitetura, ela não é, por si só, uma arquitetura.

A escolha da arquitetura, do design e dos design patterns é uma decisão que precisa levar em conta uma série de fatores, como o tipo de aplicação que se está construindo, as tecnologias utilizadas, as habilidades e conhecimentos da equipe, os prazos e o orçamento disponível. Não existe uma solução única, e a escolha mais adequada depende de um contexto específico. A compreensão da relação entre arquitetura e design, a aplicação dos princípios SOLID e o uso estratégico de design patterns são elementos chave para a construção de software robusto e eficaz.

A Taxonomia dos Design Patterns: Um Guia para Soluções de Software Reutilizáveis

Os design patterns representam um conjunto de soluções comprovadas para desafios recorrentes no desenvolvimento de software. Eles não são apenas receitas prontas, mas sim modelos que oferecem uma estrutura para resolver problemas de design de forma eficaz. A categorização desses padrões ajuda a entender suas finalidades e a aplicá-los de forma adequada. A classificação mais difundida, proveniente da obra seminal do Gang of Four (GoF) - *Design Patterns: Elements of Reusable Object-Oriented Software* -, divide os padrões em três categorias principais: padrões de criação, padrões de estrutura e padrões de comportamento. É importante ressaltar que existem outros padrões, que podem ser derivados ou encontrados em diferentes livros como *Patterns of Enterprise Application Architecture* de Martin Fowler ou *Head First Design Patterns*, que oferecem uma visão mais abrangente e variada.

Padrões de Criação: Instanciando Objetos com Flexibilidade

Os padrões de criação são focados em como os objetos são instanciados e configurados, oferecendo formas de abstrair o processo de criação, aumentando a flexibilidade, reutilização e controlando a complexidade. Padrões de criação ajudam a separar a lógica de criação da lógica de uso dos objetos, facilitando a manutenção do software e a criação de objetos com diferentes configurações.

No projeto, a criação de objetos foi abstraída através da utilização de uma *Simple Factory*, que possibilita a criação de objetos com base em uma string fornecida no momento da instanciação. Ressaltando que o *Simple Factory* não é um padrão oficial do livro do Gang of Four mas sim um passo intermediário antes da utilização de uma *Factory Method* ou *Abstract Factory* que são padrões catalogados do livro do Gang of Four.

Simple Factory

O padrão *Simple Factory* atua como um centralizador da lógica de instanciação, e comumente é implementado por um método estático, que recebe um parâmetro e devolve a instancia correspondente. A implementação deste padrão permite criar

instâncias de classes concretas com base em dados externos, como um tipo passado por parâmetro, ou um valor de um arquivo de configuração. A *Simple Factory*, no projeto, é utilizada para determinar a estratégia de geração de notas fiscais, que pode ser `CashBasisStrategy` ou `AccrualBasisStrategy` dependendo do tipo passado como parâmetro.

```
export default class InvoiceGenerationFactory {
  static create(type: string) {
    if (type === "cash") {
      return new CashBasisStrategy();
    }
    if (type === "accrual") {
      return new AccrualBasisStrategy();
    }
    throw new Error("Invalid type");
  }
}
```

Outros Padrões de Criação

Além do *Simple Factory*, existem outros padrões de criação que não foram utilizados diretamente no projeto, mas que merecem destaque:

- **Abstract Factory:**
  - **Motivação:** Oferece uma interface para criar famílias de objetos relacionados, sem especificar suas classes concretas, promovendo a criação de sistemas flexíveis que se adaptam a diferentes tipos de produtos ou recursos.
  - **Referência:** Gang of Four.
- **Factory Method:**
  - **Motivação:** Define uma interface para criação de objetos, mas permite que subclasses alterem o tipo de objetos que serão criados, promovendo a flexibilidade na criação de objetos.
  - **Referência:** Gang of Four.
- **Singleton:**
  - **Motivação:** Garante que apenas uma instância de uma classe seja criada, sendo útil para objetos que devem ter apenas um ponto de acesso em um sistema.
  - **Referência:** Gang of Four.
- **Builder:**
  - **Motivação:** Permite a construção passo a passo de objetos complexos, facilitando a criação de objetos com diferentes configurações sem a necessidade de construtores complexos.

    - **Referência:** Gang of Four.
- **Prototype:**
    - **Motivação:** Cria novos objetos através da cópia de objetos existentes (protótipos), evitando o custo de criação de objetos complexos do zero.
    - **Referência:** Gang of Four.

Padrões de Estrutura: Organizando a Complexidade

Os padrões de estrutura se concentram na forma como classes e objetos são combinados para formar estruturas maiores, mais flexíveis e reutilizáveis, auxiliando no controle da complexidade. Eles são essenciais para criar sistemas que podem ser facilmente estendidos e mantidos ao longo do tempo, permitindo o desacoplamento e a reutilização de componentes. No projeto, foram utilizados os padrões *Adapter* e *Decorator*.

Adapter

O padrão *Adapter* resolve o problema de classes com interfaces incompatíveis. Ele atua como um intermediário, convertendo a interface de uma classe em outra interface esperada pelo cliente. Isto permite que classes que não foram concebidas para trabalhar juntas colaborem sem que nenhuma delas precise de alterações em seu código original. Um exemplo prático seria a necessidade de integrar um componente legado com uma nova biblioteca.

No projeto, o *Adapter* foi utilizado em duas ocasiões. O `ExpressAdapter` converte a interface de um servidor HTTP, que no caso é o framework *Express*, em uma interface genérica, `HttpServer`, que a aplicação utiliza, e o `PgPromiseAdapter` converte as operações do *pg-promise* (uma biblioteca de acesso ao banco) para a interface genérica `DatabaseConnection`, da camada de aplicação, permitindo que a aplicação interaja com o banco sem depender diretamente da biblioteca utilizada.

```
export default class ExpressAdapter implements HttpServer {
  app: any;

  constructor() {
    this.app = express();
    this.app.use(express.json());
  }

  on(method: string, url: string, callback: Function): void {
    this.app[method](url, async function (req: any, res: any) {
      const output = await callback(req.params, req.body,
req.headers);
      res.json(output);
```

```
      });
    }

    listen(port: number): void {
      this.app.listen(port);
    }
  }

  export default class PgPromiseAdapter implements
  DatabaseConnection {
    connection: any;

    constructor() {
      this.connection = pgp()
  ("postgres://postgres:123456@localhost:5432/app");
    }

    query(statement: string, params: any): Promise<any> {
      return this.connection.query(statement, params);
    }

    close(): Promise<void> {
      return this.connection.$pool.end();
    }
  }
```

Decorator

O padrão *Decorator*, permite adicionar funcionalidades a um objeto dinamicamente, envolvendo-o em outras classes, sem alterar a estrutura original do objeto. A forma como ele é implementado garante a possibilidade de compor funcionalidades sob demanda e reutilizar decorators em diferentes cenários. Isso é muito útil em casos de implementação de logs, segurança ou caching em componentes, sem que o código original do componente necessite ser modificado.

No projeto, foi implementado o `LoggerDecorator`, que recebe um *use case* como parâmetro e adiciona funcionalidades de log, imprimindo os cabeçalhos da requisição (`User-Agent`) antes da execução do use case. Isso demonstra a capacidade do *Decorator* de adicionar comportamentos dinamicamente.

```
  import Usecase from "../usecase/Usecase";

  export default class LoggerDecorator implements Usecase {
```

```
    constructor(readonly usecase: Usecase) {}

    execute(input: any): Promise<any> {
      console.log(input.userAgent);
      return this.usecase.execute(input);
    }
  }
```

Outros Padrões de Estrutura

Além do *Adapter* e *Decorator*, existem outros padrões de estrutura que não foram utilizados diretamente no projeto, mas que merecem destaque:

- **Bridge:**
  - **Motivação:** Desacopla uma abstração da sua implementação, permitindo que ambas possam variar independentemente, o que é útil para sistemas que necessitam de flexibilidade e escalabilidade.
  - **Referência:** Gang of Four.
- **Composite:**
  - **Motivação:** Permite compor objetos em estruturas de árvore para representar hierarquias parte-todo. Ele oferece a capacidade de tratar objetos individuais e composições de forma uniforme.
  - **Referência:** Gang of Four.
- **Facade:**
  - **Motivação:** Fornece uma interface simplificada para um conjunto de interfaces mais complexas, facilitando o uso e reduzindo a complexidade de um subsistema.
  - **Referência:** Gang of Four.
- **Proxy:**
  - **Motivação:** Atua como um substituto para outro objeto, permitindo o controle do acesso, o retardamento da instanciação ou a adição de comportamentos adicionais, oferecendo uma forma de controlar ou interceptar a interação com outro objeto.
  - **Referência:** Gang of Four.

Padrões de Comportamento: O Segredo da Interação

Os padrões de comportamento se concentram em como os objetos interagem e se comunicam entre si. Eles abordam questões como a atribuição de responsabilidades, a comunicação entre objetos e o fluxo de controle em um sistema. Estes padrões promovem o desacoplamento, a modularidade, e flexibilidade, facilitando a criação de sistemas mais adaptáveis e fáceis de manter. No projeto foram utilizados os padrões *Strategy*, *Mediator* e *Presenter*.

Strategy

O padrão *Strategy* define uma família de algoritmos, encapsulando cada um deles em classes separadas e tornando-os intercambiáveis. Isso permite que o algoritmo específico utilizado possa ser selecionado em tempo de execução, oferecendo grande flexibilidade. Um exemplo claro da aplicação deste padrão é o uso de diferentes estratégias de geração de notas fiscais (`CashBasisStrategy` e `AccrualBasisStrategy`), que podem ser selecionadas dependendo do regime contábil do contrato.

```
export default interface InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[];
}

export default class CashBasisStrategy implements
InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[] {
    const invoices: Invoice[] = [];
    for (const payment of contract.getPayments()) {
      if (
        payment.date.getMonth() + 1 !== month ||
        payment.date.getFullYear() !== year
      )
        continue;
      invoices.push(new Invoice(payment.date, payment.amount));
    }
    return invoices;
  }
}

export default class AccrualBasisStrategy implements
InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[] {
    const invoices: Invoice[] = [];
    let period = 0;
    while (period <= contract.periods) {
      const date = moment(contract.date).add(period++,
"months").toDate();
      if (date.getMonth() + 1 !== month || date.getFullYear()
!== year)
```

```
          continue;
        const amount = contract.amount / contract.periods;
        invoices.push(new Invoice(date, amount));
      }
      return invoices;
    }
  }
```

Mediator

O padrão Mediator é um padrão de design comportamental que centraliza e gerencia as interações entre diversos objetos, chamados de "colegas". Em vez de que cada objeto conheça e comunique-se diretamente com os demais, criando uma complexa teia de dependências, o Mediator se coloca entre eles. Assim, sempre que um colega precisa notificar ou solicitar algo a outros objetos, ele o faz por meio do Mediator, que detém a lógica de interação e decide o que deve acontecer em resposta a cada evento. Essa abordagem reduz o acoplamento, torna o código mais flexível, e facilita manutenções e mudanças, já que alterações no comportamento de interação não exigem que todos os objetos envolvidos sejam modificados.

Em termos práticos, o Mediator coordena a colaboração entre objetos sem que eles precisem conhecer a implementação uns dos outros, apenas o Mediator. Ele age como um “controlador de tráfego”, recebendo notificações de um colega e determinando a forma correta de repassar a informação ou de desencadear ações em outros colegas. Esse padrão é especialmente útil em cenários onde o crescimento da aplicação poderia gerar um emaranhado de conexões diretas entre objetos, dificultando a manutenção e a escalabilidade do software.

No projeto apresentado, a ideia inicial era ilustrar o padrão Mediator, porém, com o decorrer do desenvolvimento, optou-se por uma abordagem mais enxuta, limitando a atuação do mediador à execução simples de eventos. Nesse sentido, a implementação resultante se assemelha mais a um “Event Bus com Observer” do que a um Mediator clássico, já que não há a coordenação estruturada e consciente entre vários componentes típicos desse padrão.

```
export default class Mediator {
  observers: { event: string; callback: Function }[];

  constructor() {
    this.observers = [];
  }

  on(event: string, callback: Function) {
    this.observers.push({ event, callback });
```

```
  }

  async publish(event: string, data: any) {
    for (const observer of this.observers) {
      if (observer.event === event) {
        await observer.callback(data);
      }
    }
  }
}
```

Ainda assim, essa escolha atendeu às necessidades imediatas do projeto, mantendo o código simples e o fluxo de eventos direto. Se, em futuras expansões, o sistema exigir coordenações mais complexas entre componentes, será possível evoluir a solução para um Mediator completo, incorporando a lógica de orquestração e controle entre objetos sem comprometer a clareza ou a manutenibilidade do código, conforme exemplo abaixo onde há uma implementação concreta de um *Mediator*:

```
// Mediator.ts (Interface do Mediator)
export interface Mediator {
  notify(sender: object, event: string, data: any):
Promise<void>;
}


// GenerateInvoices.ts (colega do Mediator)
import { Mediator } from "./Mediator";

type Input = { month: number; year: number; type: string };
type Output = { date: Date; amount: number };

export class GenerateInvoices {
  private mediator?: Mediator;

  setMediator(mediator: Mediator) {
    this.mediator = mediator;
  }

  async execute(input: Input): Promise<Output[]> {
    // Lógica de geração de faturas...
    const output: Output[] = [{ date: new Date(), amount: 6000
}];
    // Ao finalizar, notifica o Mediator
    if (this.mediator) {
```

```
      await this.mediator.notify(this, "invoicesGenerated",
output);
    }
    return output;
  }
}


// SendEmail.ts (colega do Mediator)
import { Mediator } from "./Mediator";

export class SendEmail {
  private mediator?: Mediator;

  setMediator(mediator: Mediator) {
    this.mediator = mediator;
  }

  async execute(data: any): Promise<void> {
    console.log("Enviando email sobre faturas geradas:", data);
    // Lógica de envio de email...
  }
}


// ConcreteMediator.ts (Mediator concreto que sabe a lógica do
domínio)
import { Mediator } from "./Mediator";
import { GenerateInvoices } from "./GenerateInvoices";
import { SendEmail } from "./SendEmail";

export class ConcreteMediator implements Mediator {
  constructor(
    private generateInvoices: GenerateInvoices,
    private sendEmail: SendEmail
  ) {
    // O Mediator "conecta" os colegas a si mesmo.
    this.generateInvoices.setMediator(this);
    this.sendEmail.setMediator(this);
  }

  async notify(sender: object, event: string, data: any):
Promise<void> {
    // Lógica clara: se quem enviou o evento foi
GenerateInvoices e o evento é invoicesGenerated,
```

```
    // chame SendEmail
    if (sender === this.generateInvoices && event ===
"invoicesGenerated") {
      await this.sendEmail.execute(data);
    }
  }
}

// main.ts (composição usando Mediator)
import { GenerateInvoices } from "./GenerateInvoices";
import { SendEmail } from "./SendEmail";
import { ConcreteMediator } from "./ConcreteMediator";

const generateInvoices = new GenerateInvoices();
const sendEmail = new SendEmail();
const mediator = new ConcreteMediator(generateInvoices,
sendEmail);

// O mediator conhece os objetos e a lógica de comunicação
entre eles.
generateInvoices.execute({ month: 1, year: 2022, type: "cash"
});
```

Presenter

O padrão *Presenter* é responsável por formatar e adequar dados para as necessidades específicas de cada cliente. Ele separa a lógica de apresentação da lógica de negócio, garantindo que a camada de negócio não se preocupe com os detalhes da apresentação dos dados. O *Presenter* permite transformar dados em formatos variados, como JSON, CSV, XML, entre outros, sem que haja alterações no use case que gera os dados.

No projeto, o `JsonPresenter` e o `CsvPresenter` são implementações do padrão *Presenter*, que formatam os dados de saída das notas fiscais em JSON e CSV, respectivamente.

```
export default interface Presenter {
  present(output: Output[]): any;
}
export default class JsonPresenter implements Presenter {
  present(output: Output[]): any {
    return output;
  }
}
```

```
export default class CsvPresenter implements Presenter {
  present(output: Output[]): any {
    const lines: any[] = [];
    for (const data of output) {
      const line: string[] = [];
      line.push(moment(data.date).format("YYYY-MM-DD"));
      line.push(`${data.amount}`);
      lines.push(line.join(";"));
    }
    return lines.join("\n");
  }
}
```

Outros Padrões de Comportamento

Além dos citados, existem outros padrões de comportamento que não foram utilizados diretamente no projeto, mas que merecem destaque:

- **Command:**
  - **Motivação:** Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar pedidos e suportar operações de desfazer e refazer.
  - **Referência:** Gang of Four.
- **Chain of Responsibility:**
  - **Motivação:** Permite que uma cadeia de objetos trate uma solicitação, evitando o acoplamento entre o solicitante e os tratadores.
  - **Referência:** Gang of Four.
- **Template Method:**
  - **Motivação:** Define um esqueleto de algoritmo em uma superclasse, permitindo que as subclasses redefinam certos passos do algoritmo sem modificar a estrutura geral.
  - **Referência:** Gang of Four.
- **State:**
  - **Motivação:** Permite que um objeto altere seu comportamento quando seu estado interno muda.
  - **Referência:** Gang of Four.
- **Iterator:**
  - **Motivação:** Permite que um objeto seja percorrido sem expor sua estrutura interna.
  - **Referência:** Gang of Four.
- **Observer:**

- **Motivação:** Define uma dependência um-para-muitos entre objetos, de forma que um objeto notifica automaticamente todos os seus observadores quando seu estado muda.
- **Referência:** Gang of Four.

Considerações Finais

Este resumo apresentou os principais design patterns utilizados no projeto, evidenciando suas responsabilidades e seus benefícios. O conhecimento desses padrões e a sua aplicação adequada são essenciais para o desenvolvimento de software de alta qualidade. Ao entender a classificação dos padrões, bem como os contextos em que eles podem ser mais adequados, o desenvolvedor consegue escolher as melhores abordagens e soluções para os desafios que surgem no dia a dia.

Abordagem Prática com Cenário de Notas Fiscais

Para ilustrar a aplicação dos princípios e padrões de design, é apresentado um cenário prático de emissão de notas fiscais de serviço. Este cenário envolve a implementação de diferentes regimes contábeis, como o regime de caixa, em que o reconhecimento da receita ocorre quando há o recebimento do pagamento e o regime de competência, em que o reconhecimento da receita ocorre quando o serviço é prestado, independentemente do pagamento.

DTO (Data Transfer Object)

O padrão Data Transfer Object (DTO) é um padrão fundamental na arquitetura de software, e que consiste em objetos simples, que têm apenas propriedades, com objetivo de transportar dados entre as camadas da aplicação. DTOs ajudam a desacoplar as camadas, garantindo que cada camada se preocupe apenas com os dados que são relevantes para a sua operação. Um DTO representa um contrato de comunicação entre diferentes partes de um sistema, simplificando o fluxo de informações.

Em um sistema de notas fiscais, um DTO para representar um input poderia incluir as propriedades: mês, ano e tipo (regime contábil). Já o DTO de output, poderia conter a data e o valor da nota fiscal a ser emitida.

Definição de um Use Case

Um use case descreve uma interação específica entre um usuário ou um sistema externo e o sistema sob análise. O caso de uso representa uma operação de negócio e define a entrada, a saída e a ação que será realizada.

No contexto da aplicação de notas fiscais, o caso de uso `GenerateInvoices` especifica que, dada uma entrada (mês, ano e tipo de regime), o sistema deve gerar notas fiscais

para os contratos de serviço existentes.

O Papel da Interface e da Abstração

No desenvolvimento de software, o conceito de interface e abstração desempenha um papel importante, pois ambos contribuem para a criação de sistemas modulares e flexíveis. Através de interfaces, a aplicação interage com diferentes implementações sem que seja necessário saber qual é a classe concreta sendo utilizada. Este nível de abstração permite que a aplicação se adapte mais facilmente às mudanças, sem a necessidade de modificar o núcleo da lógica.

Refatoração com Repository Pattern

A refatoração de código é uma prática essencial para melhorar a qualidade do software. O processo envolve a reestruturação do código sem alterar o seu comportamento externo, com o objetivo de aumentar a sua clareza, extensibilidade, manutenibilidade e testabilidade.

Repository Pattern

O padrão Repository tem como objetivo encapsular a lógica de acesso aos dados, isolando as camadas de negócio da camada de persistência. Em um sistema de notas fiscais, o repository seria responsável por abstrair as operações de acesso ao banco de dados, permitindo que as camadas de negócio manipulem os dados sem se preocupar com os detalhes da persistência.

O repositório se concentra na persistência de aggregates, que são grupos de entidades e objetos de valor que mantêm a consistência dos dados. Em vez de persistir cada entidade individualmente, o repositório opera em torno do aggregate, preservando as regras de negócio definidas no domínio.

É importante entender a diferença de um repository para um Data Access Object (DAO), pois um DAO opera diretamente sobre uma entidade, enquanto que um repository encapsula a persistência de aggregates, o que pode envolver mais de uma entidade.

ContractRepository e ContractDatabaseRepository

Um `ContractRepository` define a interface para a obtenção dos contratos, oferecendo um contrato para qualquer repositório concreto usar. Já o `ContractDatabaseRepository` implementa essa interface, lidando com a busca dos contratos no banco de dados e adicionando responsabilidades como realizar a conversão de dados para objetos do modelo de domínio. Este padrão desacopla a lógica de negócio das preocupações de infraestrutura.

```
export default interface ContractRepository {
  list(): Promise<Contract[]>;
}

export default class ContractDatabaseRepository implements
ContractRepository {
  constructor(readonly connection: DatabaseConnection) {}

  async list(): Promise<Contract[]> {
    const contracts: Contract[] = [];
    const contractsData = await this.connection.query(
      "select * from branas.contract",
      []
    );
    for (const contractData of contractsData) {
      const contract = new Contract(
        contractData.id_contract,
        contractData.description,
        parseFloat(contractData.amount),
        contractData.periods,
        contractData.date
      );
      const paymentsData = await this.connection.query(
        "select * from branas.payment where id_contract = $1",
        [contract.idContract]
      );
      for (const paymentData of paymentsData) {
        contract.addPayment(
          new Payment(
            paymentData.id_payment,
            paymentData.date,
            parseFloat(paymentData.amount)
          )
        );
      }
      contracts.push(contract);
    }
    return contracts;
  }
}
```

Vantagens do Repository Pattern

- **Motivação:** O padrão Repository tem como objetivo principal separar a lógica de acesso aos dados da lógica de negócio, que no dia a dia, acaba deixando os objetos de domínio muito acoplados a implementações do banco de dados, o que dificulta a manutenção e evolução do sistema.

  O padrão repository oferece diversas vantagens no desenvolvimento de software, incluindo:

  - **Separação de responsabilidades:** Isola a lógica de acesso aos dados da lógica de negócio, tornando o código mais fácil de manter e evoluir.
  - **Testabilidade:** Facilita a criação de testes unitários para a camada de negócio, pois a camada de acesso aos dados pode ser substituída por mocks ou stubs.
  - **Reutilização:** Promove a reutilização de lógica de acesso aos dados em diferentes partes da aplicação.
  - **Flexibilidade:** Permite a fácil substituição da tecnologia de persistência sem afetar outras camadas da aplicação.
  - **Referência:** Patterns of Enterprise Application Architecture de Martin Fowler.

Inversão de Dependência com o DIP (Dependency Inversion Principle)

O princípio da Inversão de Dependência (DIP) é um dos cinco princípios SOLID e é um dos mais importantes para a construção de software modular, desacoplado e flexível.

O DIP em Detalhe

O DIP preconiza que componentes de alto nível (que contém a regra de negócio) não devem depender de componentes de baixo nível (implementação de acesso à dados, por exemplo), mas sim de abstrações. Em vez de acoplar diretamente as classes concretas, este princípio promove a dependência em interfaces ou classes abstratas.

A Importância da Inversão de Dependência

O uso do DIP torna o código mais flexível, desacoplado e mais fácil de testar, e promove a criação de sistemas que podem ser alterados e evoluídos sem grandes alterações no código. Em vez de instanciar uma classe concreta, a dependência é injetada no construtor da classe. Isso promove o desacoplamento porque não força a classe a usar uma implementação específica. O controle das dependências passa a estar fora da classe em sí.

```
// Antes da inversão de dependência, o repositório era
diretamente instanciado:
// const contractRepository = new
ContractDatabaseRepository(connection);

// Após a inversão de dependência, a dependência é injetada:
export default class GenerateInvoices implements Usecase {
  constructor(
    readonly contractRepository: ContractRepository,
    readonly presenter: Presenter = new JsonPresenter(),
    readonly mediator: Mediator = new Mediator()
  ) {}
}
```

Implementação da Inversão de Dependência

A inversão de dependência é obtida através da injeção de dependência. Em vez de instanciar o `ContractDatabaseRepository` diretamente no `GenerateInvoices`, ele passa a ser injetado como parâmetro no construtor. Este padrão permite substituir o `ContractDatabaseRepository` por outras implementações do `ContractRepository`, como um repositório em memória ou um mock para testes. Com isso, o componente de alto nível depende apenas da interface e não da implementação concreta.

Adapter Pattern para Adaptar Interfaces Incompatíveis

O padrão Adapter é um padrão de projeto estrutural que possibilita a compatibilização de interfaces diferentes, permitindo que classes incompatíveis trabalhem em conjunto. Ele serve como um tradutor, convertendo uma interface em outra esperada pelo cliente. Este padrão é fundamental para criar sistemas flexíveis que podem integrar com diferentes tecnologias ou serviços sem que haja acoplamento no código.

ExpressAdapter como um Exemplo de Adapter

Um exemplo clássico de adapter é o `ExpressAdapter`, uma classe que converte a interface de um servidor HTTP, que no caso é o framework *Express*, para a interface genérica, `HttpServer`, que a aplicação utiliza, e que a aplicação espera. Esse adaptador permite que a aplicação seja executada usando o framework Express sem acoplar a sua implementação diretamente com ele. A aplicação passa a interagir apenas com a abstração `HttpServer`, sem se preocupar com os detalhes de implementação do framework.

```
export default class ExpressAdapter implements HttpServer {
  app: any;

  constructor() {
    this.app = express();
    this.app.use(express.json());
  }

  on(method: string, url: string, callback: Function): void {
    this.app[method](url, async function (req: any, res: any) {
      const output = await callback(req.params, req.body,
req.headers);
      res.json(output);
    });
  }

  listen(port: number): void {
    this.app.listen(port);
  }
}
```

DatabaseConnection como Interface para Adapters de Banco de Dados

O conceito de um `DatabaseConnection` como uma interface representa uma abstração que visa encapsular as operações de acesso ao banco de dados. Ao criar um `PgPromiseAdapter` para implementar essa interface, a aplicação pode interagir com o banco de dados através de uma abstração, em vez de depender diretamente de um framework de acesso a dados.

```
export default interface DatabaseConnection {
  query(statement: string, params: any): Promise<any>;
  close(): Promise<void>;
}

export default class PgPromiseAdapter implements
DatabaseConnection {
  connection: any;

  constructor() {
    this.connection = pgp()
("postgres://postgres:123456@localhost:5432/app");
  }
```

```
    query(statement: string, params: any): Promise<any> {
      return this.connection.query(statement, params);
    }

    close(): Promise<void> {
      return this.connection.$pool.end();
    }
  }
```

Benefícios do Adapter

- **Motivação**: O objetivo principal do Adapter é permitir a integração de diferentes partes de um sistema, mesmo quando as suas interfaces são incompatíveis, e sem que as classes precisem ser alteradas para funcionar em conjunto.
    - **Reutilização:** Permite a reutilização de componentes existentes em diferentes contextos.
    - **Flexibilidade:** Permite a fácil substituição de implementações específicas sem alterar a lógica principal do sistema.
    - **Desacoplamento:** Reduz o acoplamento entre as diferentes partes do sistema, facilitando a manutenção.
    - **Testabilidade:** Permite que a lógica da aplicação seja testada sem depender de classes concretas.
- **Referência:** Gang of Four.

Open/Closed Principle e Strategy Pattern

O Open/Closed Principle (OCP) estabelece que os módulos devem estar abertos para extensão, mas fechados para modificação. Este princípio visa garantir que a adição de novas funcionalidades não cause alterações em partes do código que já estão funcionando corretamente.

O Strategy Pattern em Ação

O Strategy Pattern define uma família de algoritmos, encapsulando cada um deles em classes separadas e tornando-os intercambiáveis. Isso permite que a seleção de um algoritmo específico seja feita em tempo de execução, com base em requisitos ou contexto da aplicação.

Em um sistema de emissão de notas fiscais, o padrão strategy é utilizado para lidar com os diferentes regimes de emissão de notas fiscais. A interface `InvoiceGenerationStrategy` define um contrato para todos os algoritmos de geração de notas fiscais e as classes `CashBasisStrategy` e `AccrualBasisStrategy`

são implementações específicas para cada tipo de regime contábil. Desta forma, o sistema se adapta dinamicamente às mudanças das regras de negócio ou necessidades de diferentes clientes, sem precisar alterar a lógica central da geração de notas fiscais.

```
export default interface InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[];
}

export default class CashBasisStrategy implements
InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[] {
    const invoices: Invoice[] = [];
    for (const payment of contract.getPayments()) {
      if (
        payment.date.getMonth() + 1 !== month ||
        payment.date.getFullYear() !== year
      )
        continue;
      invoices.push(new Invoice(payment.date, payment.amount));
    }
    return invoices;
  }
}

export default class AccrualBasisStrategy implements
InvoiceGenerationStrategy {
  generate(contract: Contract, month: number, year: number):
Invoice[] {
    const invoices: Invoice[] = [];
    let period = 0;
    while (period <= contract.periods) {
      const date = moment(contract.date).add(period++,
"months").toDate();
      if (date.getMonth() + 1 !== month || date.getFullYear()
!== year)
        continue;
      const amount = contract.amount / contract.periods;
      invoices.push(new Invoice(date, amount));
    }
    return invoices;
```

```
    }
  }
```

Simple Factory para Criar Implementações de Strategy

Para instanciar as diferentes estratégias de geração de notas fiscais, é utilizado um padrão de simple factory. A classe `InvoiceGenerationFactory` recebe um tipo de regime contábil como parâmetro e instancia a classe correspondente (`CashBasisStrategy` ou `AccrualBasisStrategy`).

```
export default class InvoiceGenerationFactory {
  static create(type: string) {
    if (type === "cash") {
      return new CashBasisStrategy();
    }
    if (type === "```typescript
accrual") {
      return new AccrualBasisStrategy();
    }
    throw new Error("Invalid type");
  }
}
```

Benefícios do OCP e Strategy Pattern

- **Motivação:** O princípio do OCP e o padrão Strategy são usados para garantir que o software seja extensível sem a necessidade de alterar o código já existente. Isso permite a fácil adição de novas funcionalidades ou comportamentos ao sistema.
    - **Extensibilidade:** Permite que novas estratégias possam ser adicionadas sem que o código existente precise ser modificado.
    - **Flexibilidade:** Facilita a alteração do comportamento do sistema em tempo de execução.
    - **Testabilidade:** Permite testar cada algoritmo separadamente.
    - **Manutenibilidade:** Reduz a complexidade e a probabilidade de erros ao modificar ou adicionar regras de negócio.
- **Referência:** OCP - Princípios SOLID, Strategy - Gang of Four.

Presenter Pattern para Formatar a Saída

O padrão Presenter se concentra na separação das responsabilidades de formatação e apresentação de dados da lógica de negócio, e garante que o use case permaneça livre

de preocupações sobre como os dados serão exibidos.

Presenter e Formatos de Saída

Em sistemas de software modernos, os dados podem precisar ser exibidos em diversos formatos (JSON, CSV, PDF, etc.) dependendo dos drivers de entrada e das necessidades dos clientes. O padrão Presenter define como essas informações podem ser adaptadas às necessidades de diferentes clientes, através de interfaces.

Em um cenário de notas fiscais, o presenter pode formatar as notas fiscais para exibição na tela, geração de arquivos para exportação para softwares contábeis e etc, sem que a regra de negócio necessite ser alterada.

```
export default interface Presenter {
  present(output: Output[]): any;
}


export default class JsonPresenter implements Presenter {
  present(output: Output[]): any {
    return output;
  }
}


export default class CsvPresenter implements Presenter {
  present(output: Output[]): any {
    const lines: any[] = [];
    for (const data of output) {
      const line: string[] = [];
      line.push(moment(data.date).format("YYYY-MM-DD"));
      line.push(`${data.amount}`);
      lines.push(line.join(";"));
    }
    return lines.join("\n");
  }
}
```

Benefícios do Presenter Pattern

- **Motivação:** A ideia central do padrão Presenter é separar a responsabilidade da lógica de negócio da lógica de apresentação ou formatação dos dados. O padrão permite que o use case se concentre em sua responsabilidade principal de obter e processar os dados, enquanto os presenters se concentram em formatar esses dados de acordo com a necessidade do cliente.

    - **Separação de Responsabilidades:** Separa a lógica de apresentação da lógica de negócio.
    - **Flexibilidade:** Facilita a adaptação da saída de dados para diferentes formatos sem alterar o core da aplicação.
    - **Reutilização:** Permite reutilizar a lógica de apresentação em diferentes partes da aplicação.
    - **Testabilidade:** Simplifica a criação de testes para cada tipo de formatação.
- **Referência:** Patterns of Enterprise Application Architecture de Martin Fowler.

Decorator Pattern para Extender Funcionalidades

O padrão Decorator permite adicionar funcionalidades a um objeto dinamicamente sem modificar a sua estrutura ou a de outras instâncias do mesmo tipo. Ele provê uma alternativa flexível para a herança, adicionando responsabilidades extras a um objeto. Decorators são muito utilizados na construção de funcionalidades transversais como logs, autorização e etc.

Decorator para Adicionar Logs a Use Cases

O padrão decorator pode ser utilizado para adicionar funcionalidades como logs a use cases sem modificar as classes originais. Criando um `LoggerDecorator` que implementa a interface do use case, é possível interceptar as chamadas ao método `execute` e adicionar logs, ao mesmo tempo em que repassa a chamada para o use case original.

```
import Usecase from "../usecase/Usecase";

export default class LoggerDecorator implements Usecase {
  constructor(readonly usecase: Usecase) {}

  execute(input: any): Promise<any> {
    console.log(input.userAgent);
    return this.usecase.execute(input);
  }
}
```

Benefícios do Decorator Pattern

- **Motivação:** O decorator tem como objetivo principal permitir que se agregue funcionalidades a um objeto sem ter que modificar o seu código original. Ele permite a criação de novas funcionalidades em tempo de execução e de forma dinâmica, sem criar novas subclasses e evitando, dessa forma, o aumento da complexidade do sistema.

    - **Extensibilidade:** Permite adicionar funcionalidades em tempo de execução sem alterar o código do objeto original.
    - **Flexibilidade:** Facilita a combinação de diferentes funcionalidades de forma dinâmica.
    - **Reutilização:** Permite reutilizar decorators em diferentes objetos.
    - **Manutenibilidade:** Reduz a complexidade do código e a probabilidade de erros.
- **Referência:** Gang of Four.

Controller, Composition Root e Mediator Pattern

Para finalizar o ciclo de estudo dos padrões de projeto, são abordados os padrões Controller, Composition Root e Mediator, que garantem o bom funcionamento da aplicação, tanto no ponto de vista da orquestração de requisições, como da configuração do sistema e comunicação entre componentes.

Controller Pattern para Interação com o Driver

O padrão Controller serve como um intermediário entre o driver e a aplicação, gerenciando a entrada e a saída de dados, conectando o mundo externo com o sistema. Ele recebe dados da solicitação, os encaminha para a aplicação e formata a resposta para o driver que fez a chamada. Um controlador pode ser um serviço HTTP, uma fila de mensagens ou qualquer tipo de interface que serve para interagir com o sistema.

No exemplo prático, o `MainController` recebe a requisição da API, extrai os parâmetros do corpo da requisição (body), e encaminha essa informação ao use case `GenerateInvoices`, retornando o resultado formatado ao cliente, via o `HttpServer`.

```
export default class MainController {
  constructor(readonly httpServer: HttpServer, readonly
usecase: Usecase) {
    httpServer.on(
      "post",
      "/generate_invoices",
      async function (params: any, body: any, headers: any) {
        const input = body;
        body.userAgent = headers["user-agent"];
        body.host = headers.host;
        const output = await usecase.execute(input);
        return output;
      }
    );
```

```
    }
  }
```

Composition Root para a Configuração da Aplicação

O Composition Root é um local centralizado na aplicação onde as dependências são configuradas e as instâncias dos objetos são criadas. Ele é o ponto de entrada da aplicação e tem a responsabilidade de compor o grafo de dependências, montando o encadeamento de objetos de modo correto.

Com a criação do main.ts, é demonstrado como o Composition Root serve como ponto de configuração, onde as dependências são conectadas. Isto possibilita a criação das instâncias do repository, do mediator, do use case e outros serviços, que são então passadas ao controller, ao HTTP Server e aos demais objetos que necessitam dessas dependências.

```
import ContractDatabaseRepository from
"./infra/repository/ContractDatabaseRepository";
import ExpressAdapter from "./infra/http/ExpressAdapter";
import GenerateInvoices from
"./application/usecase/GenerateInvoices";
import JsonPresenter from "./infra/presenter/JsonPresenter";
import LoggerDecorator from
"./application/decorator/LoggerDecorator";
import MainController from "./infra/http/MainController";
import PgPromiseAdapter from
"./infra/database/PgPromiseAdapter";
import Mediator from "./infra/mediator/Mediator";
import SendEmail from "./application/usecase/SendEmail";

const connection = new PgPromiseAdapter();
const contractRepository = new
ContractDatabaseRepository(connection);
const mediator = new Mediator();
const sendEmail = new SendEmail();
mediator.on("InvoicesGenerated", async function (data: any) {
  await sendEmail.execute(data);
});
const generateInvoices = new LoggerDecorator(
  new GenerateInvoices(contractRepository, new JsonPresenter(),
mediator)
);
const httpServer = new ExpressAdapter();
```

```
new MainController(httpServer, generateInvoices);
httpServer.listen(3000);
```

Outros Padrões de Comportamento

Além dos padrões *Strategy*, *Mediator* e *Presenter*, existem outros padrões de comportamento que não foram utilizados diretamente no projeto, mas que merecem destaque:

- **Command:**
  - **Motivação:** Encapsula uma solicitação como um objeto, permitindo parametrizar clientes com diferentes solicitações, enfileirar pedidos e suportar operações de desfazer e refazer.
  - **Referência:** Gang of Four.
- **Chain of Responsibility:**
  - **Motivação:** Permite que uma cadeia de objetos trate uma solicitação, evitando o acoplamento entre o solicitante e os tratadores.
  - **Referência:** Gang of Four.
- **Template Method:**
  - **Motivação:** Define um esqueleto de algoritmo em uma superclasse, permitindo que as subclasses redefinam certos passos do algoritmo sem modificar a estrutura geral.
  - **Referência:** Gang of Four.
- **State:**
  - **Motivação:** Permite que um objeto altere seu comportamento quando seu estado interno muda.
  - **Referência:** Gang of Four.
- **Iterator:**
  - **Motivação:** Permite que um objeto seja percorrido sem expor sua estrutura interna.
  - **Referência:** Gang of Four.
- **Observer:**
  - **Motivação:** Define uma dependência um-para-muitos entre objetos, de forma que um objeto notifica automaticamente todos os seus observadores quando seu estado muda.
  - **Referência:** Gang of Four.

Organização do Projeto em Camadas: Uma Estrutura para a Manutenção e Evolução

A organização de um projeto de software em camadas é uma prática fundamental para garantir a clareza, modularidade e manutenibilidade do código. Ao separar as responsabilidades em camadas bem definidas, o sistema torna-se mais fácil de

entender, modificar e testar. O projeto em questão, assim como muitos outros, é estruturado em três camadas principais: domain, application e infrastructure.

A Camada Domain: O Coração da Lógica de Negócio

A camada domain concentra as entidades e os objetos de valor, que representam a lógica de negócio do sistema. Aqui, são definidos os conceitos, as regras e os comportamentos específicos do problema que o software busca resolver. As classes dentro desta camada não devem depender de detalhes de infraestrutura ou de implementação, e sim de conceitos abstratos que pertencem ao domínio do problema, como as classes `Contract`, `Payment` e `Invoice` apresentadas no exemplo prático. Essa separação permite que a lógica de negócio seja modificada sem que haja impacto nas camadas mais externas. A camada de domínio reflete o "vocabulário" do negócio, facilitando o entendimento do sistema pelos envolvidos e especialistas do domínio.

A Camada Application: Orquestrando os Casos de Uso

A camada application tem a responsabilidade de orquestrar a execução dos casos de uso do sistema, coordenando as ações necessárias para atender às requisições dos clientes. Ela não contém a lógica de negócio em si, mas sim o fluxo das operações, chamando os componentes do domínio e coordenando a execução, e aplicando as decisões de design para direcionar as chamadas para a correta classe, como o `GenerateInvoices` que coordena a criação das notas fiscais, utilizando o `ContractRepository` para obter os dados e o `Presenter` para formatar a saída, orquestrando o fluxo do processo, e delegando tarefas específicas às camadas adequadas. Nesta camada, as decisões de design de software são orquestradas e definidas, e é o local onde o TDD é mais efetivo, por ser a camada central e mais fácil de ser testada.

A Camada Infrastructure: Conectando o Software ao Mundo Externo

A camada infrastructure contém a implementação dos frameworks, bibliotecas e detalhes de conexão com o mundo externo. Ela é responsável por lidar com a persistência de dados, a comunicação com serviços externos, a interface com o usuário, a criação de logs, e a demais ações que necessitam de interfaces de baixo nível. O objetivo desta camada é desacoplar a aplicação dos detalhes de implementação específicos, permitindo a fácil substituição da tecnologia ou do framework sem afetar as outras camadas. Nesta camada encontram-se o `ExpressAdapter` que implementa o `HttpServer`, o `PgPromiseAdapter` que implementa o `DatabaseConnection`, o `JsonPresenter` e o `CsvPresenter` que implementam a interface `Presenter` e outras classes responsáveis pela comunicação com recursos externos.

Considerações Finais

Neste capítulo, foram abordados os princípios SOLID, design patterns e suas aplicações em um cenário prático de emissão de notas fiscais, mostrando como as diferentes camadas do software podem ser testadas com diferentes estratégias de teste. Os testes de unidade, assim como os testes de integração, garantem que o código está funcionando corretamente e que as mudanças feitas não vão gerar problemas de regressão, o que é fundamental no processo de desenvolvimento de software.

É crucial o entendimento de que os padrões de design não devem ser aplicados de forma arbitrária ou automática, mas sim quando houver uma necessidade clara e um problema específico a ser resolvido. A sobre utilização de padrões de design, pode levar a sistemas complexos com camadas de abstração desnecessárias que tornam o sistema difícil de entender e manter. Muitos desenvolvedores, empolgados com o aprendizado de design patterns, podem cair na armadilha da "patternite", que se caracteriza pela tendência de aplicar padrões em todas as situações, mesmo quando não há uma real necessidade. Isso pode gerar complexidades e acoplamentos excessivos no projeto, o que compromete a sua qualidade. A aplicação de um padrão, portanto, precisa estar alinhada com as necessidades e complexidade do software que está sendo implementado, buscando sempre o equilíbrio e a adequação entre abstração e simplicidade.

Os princípios SOLID, por sua vez, estão diretamente ligados aos design patterns e devem ser seguidos em conjunto para construir software de alta qualidade, pois o design patterns quando usado de forma consciente, ajudam a fortalecer os princípios SOLID. Cada um dos princípios apresentados nesse material tem relação direta com os padrões que foram utilizados. O padrão *Repository*, por exemplo, tem como objetivo o *Single Responsibility Principle* (SRP), separando as responsabilidades da lógica de negócios e do acesso aos dados, ao passo que o padrão *Strategy*, aplicado juntamente com a interface, e com a *Simple Factory*, respeita o *Open/Closed Principle* (OCP) ao permitir que novas estratégias possam ser facilmente adicionadas sem modificar o código já existente. O *Dependency Inversion Principle* (DIP), por sua vez, é aplicado através da injeção de dependências, que permite a desacoplar objetos de alto nível das suas implementações. A combinação desses conceitos, em conjunto com design patterns, ajuda a construir um sistema coeso, flexível e facilmente adaptável a novas funcionalidades ou cenários.

Os design patterns que utilizamos neste projeto são apenas uma pequena amostra do universo de possibilidades de soluções de design existentes, o que se reflete em diversos livros clássicos na literatura como o *Gang of Four*, *Patterns of Enterprise Application Architecture*, e *Enterprise Integration Patterns*, entre outros. Esses padrões e outros diversos, foram sendo identificados ao longo dos anos por diversos desenvolvedores, que buscaram solucionar problemas comuns, e que acabaram se tornando uma ferramenta para organizar os componentes do software de forma reutilizável e desacoplada, facilitando a construção e a manutenção do software.

Referências

Livros, artigos e links relevantes que aprofundam os conceitos e padrões abordados:

Livros

- **Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John.** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
    - Conhecido como o livro do "Gang of Four" (GoF), é uma referência fundamental no estudo de design patterns. Apresenta uma ampla variedade de padrões de criação, estrutura e comportamento, que se tornaram a base para muitos outros trabalhos na área.
- **Fowler, Martin.** *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
    - Um guia prático para projetar sistemas empresariais complexos, apresentando padrões de arquitetura e design que auxiliam a construir aplicações robustas e flexíveis. Este livro aborda padrões para lidar com a persistência de dados e o acoplamento entre diferentes camadas.
- **Freeman, Eric; Freeman, Elisabeth; Sierra, Kathy; Bates, Bert.** *Head First Design Patterns*. O'Reilly Media, 2004.
    - Uma abordagem didática e prática para o aprendizado de design patterns. Este livro utiliza uma linguagem mais acessível, com exemplos e exercícios, para facilitar a compreensão e memorização dos padrões.
- **Hohpe, Gregor; Woolf, Bobby.** *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.
    - Um livro focado na integração de sistemas empresariais, explorando padrões de mensageria e comunicação entre diferentes aplicações e serviços. Este livro aborda a comunicação de forma assíncrona entre diferentes partes do sistema.
- **Martin, Robert C.** *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
    - Um guia para escrever código limpo e fácil de manter. O livro aborda práticas de programação, design de classes e organização de projetos para melhorar a qualidade e legibilidade do código.
- **Martin, Robert C.** *Clean Architecture: A Craftsman's Guide to Software Structure*. Prentice Hall, 2017.
    - Este livro apresenta os princípios da arquitetura limpa, que visa criar sistemas independentes de frameworks, tecnologias de persistência e interfaces com o usuário, focando no modelo de domínio do sistema.

Artigos e Links

- **SOLID Principles:**
    - https://blog.cleancoder.com/uncle-bob/2020/10/18/Solid-Relevance.html

        - Artigo do blog Clean Coder com a descrição detalhada dos princípios SOLID e sua importância no desenvolvimento de software.
- **Design Patterns Guru:**
    - https://refactoring.guru/design-patterns
        - Site com descrições detalhadas, diagramas e exemplos práticos de diversos design patterns. O site também aborda os princípios SOLID.
- **Martin Fowler:**
    - https://martinfowler.com/
        - Site de Martin Fowler com diversos artigos e livros sobre arquitetura, design e padrões de software.

Resumo do capítulo

Aulas



Introdução 16:07

Criando o contexto do projeto 11:24

DTO (Data Transfer Object) 18:05

Repository 09:22

Principio DIP (Dependency Inversion Principle) 06:19

Adapter 23:54

Princípio OCP (Open Closed Principle) e Strategy Pattern 24:32

Presenter 09:23

Decorator 19:18

Controller e Composite Root 14:49

Mediator 10:40

Considerações finais 16:18

Código fonte